

Abstract

Penetration testing, the authorised simulation of adversarial attacks, remains inherently resistant to full automation. This stems from two structural limitations: traditional tools are constrained by signature-based detection that fails to generalise, while LLM-based autonomous agents, despite exhibiting reasoning capabilities, degrade under long-horizon and state-intensive conditions.

This thesis presents a critical analysis of automated and AI-driven approaches across four dimensions: reasoning, autonomy, adaptability, and memory management. Particular focus is given to LLM-based agents, examining paradigms such as ReAct, Chain-of-Thought, and tool-augmented reasoning, and exposing the unresolved tension between planning and execution. Existing memory mechanisms—including in-context storage and retrieval-augmented generation—are shown to be insufficient for sustained operations.

By modelling penetration testing as a long-horizon decision process under uncertainty and adversarial constraints, the study identifies fundamental systemic limitations and derives a structured research gap taxonomy. It concludes by defining the core properties required for robust next-generation autonomous systems.

Keywords: Autonomous Penetration Testing, AI Agents, LLM, Cybersecurity.

Résumé

Les tests de pénétration constituent la simulation autorisée et systématique d'attaques adversariales dans des conditions contrôlées. Malgré des décennies d'investissements technologiques, cette discipline résiste obstinément à une automatisation exhaustive. Deux modes de défaillance structurels s'affrontent : les outils automatisés traditionnels sont limités par des paradigmes rigides de reconnaissance de motifs, tandis que les agents autonomes basés sur les grands modèles de langage (LLM), bien que dotés de capacités de raisonnement sémantique, s'effondrent face aux exigences cognitives des évaluations prolongées.

Cette thèse présente une analyse critique et rigoureuse de trois générations d'approches automatisées, structurée autour de quatre dimensions comparatives : raisonnement, autonomie, adaptabilité et gestion de la mémoire d'état. L'accent est mis sur les agents LLM, dont les paradigmes fondamentaux—boucle ReAct, chaîne de pensée, exploration par arbre de pensées—sont déconstruits méthodiquement pour exposer la tension persistante entre planification délibérative et exécution réactive.

Cette recherche identifie les conditions précises dans lesquelles chaque approche est structurellement insuffisante et cristallise ses conclusions en une taxonomie de six lacunes de recherche justifiées. La thèse conclut en définissant les propriétés théoriques que les futurs systèmes autonomes doivent satisfaire pour dépasser le plafond de performance actuel.

Mots-clés : Tests d'intrusion autonomes, Agents IA, LLM, Cybersécurité, Lacunes de recherche.

Contents

Contents	iii
List of Tables	iv
List of Tables	iv
List of Acronyms	v
1 Introduction	1
1.1 Scientific and Societal Context	1
1.2 The Automation Challenge	1
1.3 The LLM Promise and Its Limits	2
1.4 Research Problem Statement	2
1.5 Research Questions	2
1.6 Research Hypotheses	3
1.7 Scope and Document Structure	3
2 Theoretical Background	4
2.1 Penetration Testing: Practice, Standards, and Cognitive Demands	4
2.1.1 The Vulnerability Landscape and Its Detection Paradigms	5
2.1.2 Vulnerability Chaining: The Expert Tester’s Core Skill	6
2.2 Large Language Models: Relevant Properties for Autonomous Agents	7
2.2.1 Emergent Security Knowledge	7
2.2.2 In-Context Learning and Adaptation	8
2.2.3 The Context Window and the “Lost in the Middle” Problem	8
2.3 The Decision-Making Nature of Penetration Testing	8
3 State of the Art: Critical Comparative Analysis	10
3.1 Comparison Framework	10
3.2 First Generation: Traditional Automated Scanning	11
3.2.1 Signature-Based Dynamic Testing	11

3.2.2	Heuristic Active Scanning	12
3.2.3	Domain-Specific Expert Tools: SQLMap as a Case Study	13
3.2.4	Summary: Why the Template Ceiling Cannot Be Crossed from Within	13
3.3	Second Generation: Rule-Based Automation and Expert Systems	14
3.4	Third Generation: AI-Driven Autonomous Agents	15
3.4.1	Demonstrated Capabilities: Establishing the Baseline	15
3.4.2	LLM-Based Agent Reasoning Paradigms: A Deep Analysis	15
3.4.3	Memory Mechanisms in LLM-Based Security Agents	19
3.4.4	Critical Analysis of Deployed LLM Security Agents	19
3.4.5	Cross-Cutting Analysis: What the Convergence of Failures Reveals	21
4	Synthesis: Causal Mechanisms of Convergent Failure	23
4.1	Comprehensive Cross-Generational Comparison	23
4.2	Causal Mechanism 1: The Information Bottleneck	25
4.3	Causal Mechanism 2: Implicit State Representation	26
4.4	Causal Mechanism 3: Unverified Inference Propagation	27
4.5	Compound Degradation: How the Three Mechanisms Interact	28
5	Research Gap Taxonomy	29
5.1	Gap Taxonomy	29
5.2	Gap Interactions and Compounding Effects	32
6	Conceptual Perspective	34
6.1	The External State Principle	34
6.2	The Verified Evidence Principle	35
6.3	The Hierarchical Decomposition Principle	35
7	Discussion	37
7.1	Implications for Cybersecurity Practice and Governance	37
7.2	Limitations of This Analysis	37
7.3	Open Challenges and Research Risks	38
8	Conclusion	39
8.1	Summary of Findings	39

8.2	Evaluation of Research Hypotheses	39
8.3	The Research Gap Taxonomy	40
8.4	Research Agenda and Closing Remarks	40
	Bibliography	41

List of Tables

2.1	OWASP Top 10 (2021) Classified by Detection Paradigm	6
3.1	LLM-Based Security Agents: Comparative Dimensional Analysis	21
4.1	Comprehensive Cross-Generational Comparison of Automated Penetration Testing Approaches	24
5.1	Research Gap Taxonomy: Evidence Map and Interaction Matrix	32

List of Acronyms

AI	Artificial Intelligence	OWASP	Open Web Application Security Project
APT	Advanced Persistent Threat		
CAPEC	Common Attack Pattern Enumeration and Classification	RAG	Retrieval-Augmented Generation
		RCE	Remote Code Execution
CoT	Chain-of-Thought	ReAct	Reason + Act
CVE	Common Vulnerabilities and Exposures	SAST	Static Application Security Testing
CVSS	Common Vulnerability Scoring System	SSRF	Server-Side Request Forgery
		SSTI	Server-Side Template Injection
CWE	Common Weakness Enumeration	SQLi	SQL Injection
DAST	Dynamic Application Security Testing	ToT	Tree of Thoughts
IDOR	Insecure Direct Object Reference	TTP	Tactic, Technique, and Procedure
LLM	Large Language Model		
OAST	Out-of-band Application Security Testing	WAF	Web Application Firewall
		XSS	Cross-Site Scripting

Chapter 1 Introduction

1.1 Scientific and Societal Context

Every networked software system represents a potential attack surface. As organisations accelerate deployment through cloud-native architectures and continuous delivery, the complexity of their digital estates outpaces the security community’s capacity to verify it. IBM’s 2024 Cost of a Data Breach report recorded an average breach cost of \$4.88 million USD per incident (IBM Security, 2024), while the ISC2 Workforce Study documents a global shortage of four million qualified security professionals (ISC2, 2023). These figures are symptoms of a structural mismatch between deployment velocity and security assessment capacity.

Against this backdrop, penetration testing—the authorised simulation of adversarial attack under controlled conditions—has established itself as the most rigorous mechanism for proactive risk assessment. A skilled tester does not merely scan; they reason, form hypotheses, adapt strategy in response to observed evidence, and determine through actual exploitation whether a weakness can be leveraged to cause real harm. This expert reasoning is precisely what makes penetration testing valuable and precisely what makes it expensive, slow, and impossible to scale at the pace the industry demands.

1.2 The Automation Challenge

The automation challenge in penetration testing is not simply a question of scale but of *kind*. The vulnerability classes that cause the most damaging breaches are precisely those most resistant to automation. Broken access control (OWASP A01 (OWASP Foundation, 2021)) requires understanding what a user *should* and *should not* be permitted to access—a condition that varies per application and cannot be encoded as a universal pattern. Business logic flaws require understanding the intended workflow; tools that probe input fields with injection payloads are structurally blind to this class of vulnerability.

These examples explain why the first generation of automated tools—despite decades of development—has not replaced expert human assessment and frame the central question: can the second generation, built on Large Language Models capable of semantic reasoning, succeed where the first generation fundamentally could not?

1.3 The LLM Promise and Its Limits

Large Language Models, trained on vast corpora including security research, vulnerability databases, and exploit documentation, offer a qualitatively different possibility. They can reason about novel situations, transfer knowledge across domains, and generalise beyond memorised patterns. Academic research confirms this capability: Liu et al. (2024) demonstrated 87% autonomous CVE exploitation with structured context (Liu et al., 2024); Deng et al. (2023) showed a structured agent completing $\sim 67\%$ of HackTheBox challenges vs. 14.3% for an unaugmented model (Deng et al., 2023); Happe and Cito (2023) demonstrated autonomous privilege escalation across target Linux hosts (Happe and Cito, 2023). These results establish that LLMs have crossed the template ceiling.

Yet every research group building these systems reports the same structural breakdowns: agents revisit surfaces already exhausted, lose multi-step attack chains in context compression events, and cannot distinguish confirmed from hallucinated findings. The pattern is consistent: the models have the capability; the architectures surrounding them do not provide the infrastructure to sustain it. The limit is not LLM intelligence—it is agent architecture.

1.4 Research Problem Statement

This thesis investigates: *What are the precise, structurally identifiable causes of autonomous penetration testing failure, and what theoretical properties must future systems possess to address them?* This asks not whether LLM-based agents can perform penetration testing—that is answered affirmatively—but why the gap between demonstrated short-horizon capability and sustained long-horizon performance is structural, not quantitative.

1.5 Research Questions

- RQ1** What structural properties of traditional automated scanning prevent generalisation beyond pre-catalogued patterns, and why cannot engineering improvements overcome them?
- RQ2** Through what causal mechanisms do LLM-based agent architectures—reasoning loops, tool-augmentation strategies, and memory approaches—produce the convergent failure modes observed across evaluated systems?
- RQ3** What theoretical properties—memory persistence, hypothesis lifecycle management, finding-relationship representation, and safety enforcement—are necessary for an autonomous system to maintain strategic coherence across a full engagement?

1.6 Research Hypotheses

H1 – Architecture Primacy: The primary constraint on LLM-based penetration testing agent performance is not model capability but agent architecture. The inability to maintain domain-structured state external to the context window is the binding constraint preventing generalisation to sustained real-world engagement.

H2 – Convergent Failure: The failure modes observed across PentestGPT, Hacking-BuddyGPT, and AutoAttacker are not independent system-specific bugs but manifestations of a common set of structural absences that any system sharing the trajectory-as-memory model will exhibit.

1.7 Scope and Document Structure

This thesis is a research-driven theoretical analysis. It presents no software implementation, experimental dataset, or benchmarking sequence. All claims are supported by cited research or explicit logical inference. The scope is limited to automated web application and host-based penetration testing.

Chapter 2 establishes theoretical foundations. Chapter 3 presents the core comparative analysis across three generations of automated approaches. Chapter 4 synthesises causal failure mechanisms. Chapter 5 presents the formal research gap taxonomy. Chapter 6 articulates required theoretical properties. Chapter 7 examines broader implications. Chapter 8 synthesises conclusions and the research agenda.

Chapter 2 Theoretical Background

This chapter establishes the conceptual vocabulary and foundational understanding necessary to engage with the comparative analysis in Chapter 3. The exposition is deliberately focused: only concepts that are directly required for the analytical claims that follow are introduced, and each concept is introduced at the level of precision needed for those claims—not as a tutorial, not as a survey.

2.1 Penetration Testing: Practice, Standards, and Cognitive Demands

Definition — Penetration Testing. Penetration testing is the authorised, systematic simulation of adversarial attack against a target system, conducted by a skilled practitioner under controlled conditions with the explicit objective of identifying exploitable vulnerabilities and providing concrete evidence of their real-world impact. It is distinguished from vulnerability scanning by the requirement to confirm exploitability through actual exploitation attempts, rather than probabilistic inference from detected indicators.

This definition implies something important: penetration testing is not a static analysis exercise. It is an interactive, evidence-driven investigation in which each action informs the next. The practitioner forms a hypothesis (“this endpoint may not enforce user-level access control”), tests it (by issuing a request with a different user’s identifier), interprets the result (a successful response confirms the hypothesis; a 403 or redirect refutes it), and revises their strategy accordingly. This Hypothesis–Test–Interpret cycle, repeated hundreds of times across a complex application, constitutes the cognitive core of expert penetration testing. It is a form of empirical inquiry in a hostile, partially observable environment.

The Penetration Testing Execution Standard (PTES, 2012) defines a seven-phase engagement lifecycle: pre-engagement (scope and authorisation), intelligence gathering, threat modelling, vulnerability identification, exploitation, post-exploitation, and reporting. Each phase places different cognitive demands on the practitioner. Intelligence gathering is predominantly a search and synthesis task. Threat modelling requires understanding the target’s business context well enough to identify what an attacker would most want to achieve. Vulnerability identification requires both broad coverage (not missing anything testable) and contextual relevance (not testing things that cannot possibly lead to impact). The exploitation phase is the most cognitively demanding: it requires converting a theoretical weakness into a working attack under real-world defensive conditions. Post-

exploitation requires reasoning about the implications of a successful breach: what further access does this enable? What data is now reachable? What persistence mechanisms can be established?

The automation challenge is sharpest at exploitation and post-exploitation. These phases require not knowledge of generic vulnerability classes, but reasoning about the *specific* instance of a vulnerability in the *specific* application under test, given the *specific* defensive infrastructure it operates behind.

2.1.1 The Vulnerability Landscape and Its Detection Paradigms

Understanding why automation founders on the exploitation phase requires examining the diversity of what “vulnerability” means across the landscape of modern web applications. The OWASP Top 10 (2021) (OWASP Foundation, 2021) enumerates the most prevalent and impactful vulnerability classes, and a careful analysis reveals a fundamental division that is directly relevant to automation.

Some vulnerability classes are detectable primarily through pattern matching. Outdated components with known CVEs can be identified by comparing version strings against a vulnerability database. Missing security headers can be detected by checking HTTP response headers against a required list. Default credentials can be tested by trying known username-password pairs from published lists. These are tasks at which automated tools excel, and where automation provides genuine, measurable value.

Other vulnerability classes are detectable only through semantic reasoning about application intent. Broken access control (the most prevalent category, ranked A01) requires understanding who is permitted to access what, which varies per application and is never universally specifiable. Insecure design (A04) describes architecturally flawed security logic that can only be identified by someone who understands what the correct design should have been. Server-Side Request Forgery as a business-logic variant (A10 in specific deployments) requires understanding that the application’s server-side fetch capability is being exploited, which depends on recognising that the application fetches URLs at all and that those URLs are attacker-influenced. All three of these categories demand that the tester model the application’s intended behaviour and recognise that its actual behaviour departs from it in a security-relevant way. This is semantic reasoning that no template or heuristic rule can encode universally.

Table 2.1: OWASP Top 10 (2021) Classified by Detection Paradigm

Rank	Category	Description	Detection Mode
A01	Broken Access Control	Users access unauthorised resources via privilege failure	Semantic
A02	Cryptographic Failures	Sensitive data transmitted or stored without encryption	Configuration
A03	Injection	Attacker-controlled data interpreted as commands	Pattern + Semantic
A04	Insecure Design	Architecturally flawed security logic	Semantic only
A05	Security Misconfiguration	Default credentials, exposed debug interfaces, verbose errors	Pattern
A06	Outdated Components	Vulnerable third-party libraries with known CVEs	CVE database
A07	Authentication Failures	Weak session management, credential bypass logic	Pattern + Semantic
A08	Data Integrity Failures	Unsigned serialised objects, compromised update mechanisms	Semantic
A09	Logging Failures	Insufficient detection capacity (inferred from absence)	Inference
A10	SSRF	Server fetches attacker-controlled URLs	Semantic

The classification in Table 2.1 reveals a structural truth: the majority of the most impactful vulnerability categories require reasoning beyond pattern recognition. This is not an artefact of the particular taxonomy; it reflects the reality that the most dangerous flaws in modern software are those that emerge from the gap between intended and actual behaviour in complex, context-dependent systems—a gap that is definitionally inaccessible to systems that have no model of intended behaviour.

2.1.2 Vulnerability Chaining: The Expert Tester’s Core Skill

Beyond individual vulnerability detection, the practice that most distinguishes expert human testers is *vulnerability chaining*: composing multiple individually limited findings into a high-impact exploitation path. An expert discovering SSRF does not simply record it; they reason about what internal services are reachable, whether cloud metadata

credentials are accessible, and whether those credentials enable further lateral movement. This chain—SSRF to metadata to credentials to storage exfiltration—crosses multiple vulnerability categories and has critical severity even though the initial finding might be rated medium. A tool that records SSRF in isolation and moves to the next endpoint will never recover the chain. Maintaining the strategic context across hundreds of subsequent actions is precisely the kind of reasoning current automated tools cannot sustain.

2.2 Large Language Models: Relevant Properties for Autonomous Agents

Large Language Models are trained to predict the next token in a sequence, given all preceding tokens. Through training on sufficiently large and diverse corpora, models at scale develop capabilities that were neither explicitly trained nor anticipated: the ability to reason through multi-step problems, to recognise structural analogies across domains, to generate contextually appropriate code, and to adapt their output style and strategy to instructions provided in the input. These "emergent" capabilities are the foundation of their relevance to penetration testing.

2.2.1 Emergent Security Knowledge

The training corpora of frontier LLMs include published security research, vulnerability databases, penetration testing guides, bug bounty write-ups, and exploit documentation. Through exposure to this material at scale, the models develop a latent representation of how vulnerabilities manifest, what indicators distinguish vulnerable from non-vulnerable behaviour, and how exploitation techniques are applied in practice. This is not the same as memorising templates; it is a generalised understanding that can be applied to instances the model has never seen in exactly that form.

The practical significance of this is confirmed by Liu et al.'s (2024) result: a frontier LLM, given a structured CVE description as context, could autonomously exploit 87% of a diverse set of one-day vulnerabilities without any template or hard-coded exploit logic (Liu et al., 2024). Critically, the same model without the structured CVE context achieved only approximately 7% success—demonstrating that the security knowledge is present within the model but requires structured scaffolding to be reliably channelled. This distinction between the model's intrinsic capability and its scaffolded performance is central to the analysis in Chapter 3.

2.2.2 In-Context Learning and Adaptation

Unlike traditional software, which behaves identically on identical inputs regardless of what preceded them in the session, LLMs exhibit *in-context learning*: the model’s behaviour at any given step is conditioned by the entire preceding context window. Instructions, examples, and observations provided earlier in the conversation shape how the model reasons about and responds to subsequent inputs—without any change to the model’s weights.

This property makes LLMs naturally suited to the iterative, evidence-driven structure of penetration testing: each observation can be incorporated into the model’s contextual understanding, updating its assessment of the target and shaping subsequent strategy. The limitation is that this context-dependence has a hard boundary: the context window.

2.2.3 The Context Window and the “Lost in the Middle” Problem

Every interaction with an LLM is bounded by a maximum context length, measured in tokens (roughly corresponding to word fragments). Information outside this window is completely inaccessible to the model—not degraded, not compressed, but absent. Within the window, information is not accessed with uniform reliability. Liu et al. (2023) characterised the “lost in the middle” phenomenon empirically: model performance on retrieval tasks degrades substantially when the target information is positioned in the middle of a long context, with the model showing significantly stronger recall for information at the beginning (primacy) and at the end (recency) of the input Liu et al. (2023).

For penetration testing agents, this creates a compounding problem. As the session progresses, the context window fills with tool outputs—verbose HTTP responses, scan results, error messages—that may each be individually necessary to retain but collectively exceed the window. The model must either truncate old content (losing early findings), summarise it (losing specificity), or accept that early findings will be effectively inaccessible due to attention dilution even if they technically remain in the window. Any of these outcomes can cause the agent to lose the thread of a multi-step investigation.

2.3 The Decision-Making Nature of Penetration Testing

Penetration testing is a sequential decision problem in which an agent must select actions from a large set of possibilities, observe the results, and use those results to inform subsequent choices (Russell and Norvig, 2020). Four compounding properties shape the challenge:

High-dimensional, partially observable state: The agent can only observe target responses to probes; the true vulnerability state is never directly visible. Inferences from observations are always uncertain.

Sparse and deferred rewards: The vast majority of probing actions produce uninformative results. The significance of a finding may only become clear much later when connected to other findings.

Adversarial information suppression: WAFs strip error messages, rate limiters make timing ambiguous, and custom error pages conceal status codes. The agent must triangulate truth from patterns across many observations.

Long-horizon dependency: The most valuable exploitation paths connect findings separated by many actions. Maintaining awareness of early-session findings throughout a multi-hour engagement is the defining cognitive challenge—and the primary point of failure for current systems.

Chapter 3 State of the Art: Critical Comparative Analysis

This chapter constitutes the core scientific contribution of this thesis. It analyses three generations of automated penetration testing approaches through a consistent comparative framework, with particular depth directed at the LLM-based agent generation that defines the current research frontier. The objective is not to describe what systems exist, but to explain—causally and with evidential support—why each approach exhibits the capabilities and limitations it does.

3.1 Comparison Framework

Four analytical dimensions are applied consistently across all approaches examined in this chapter:

- **Reasoning Capability:** Can the system infer the probable presence of a vulnerability from contextual evidence, without relying on a pre-encoded signature for that specific vulnerability type? This dimension distinguishes systems that recognise from systems that reason.
- **Autonomy Level:** To what extent does the system self-direct its action sequence throughout a session, without human intervention to resolve ambiguities, redirect strategy, or interpret ambiguous findings?
- **Adaptability:** Does the system modify its strategy in response to target-specific evidence encountered during the session? Is this adaptation rule-based (branching along pre-programmed paths) or evidence-driven (forming new hypotheses from unexpected observations)?
- **State-Memory Handling:** How does the system maintain its knowledge of what it has done, what it has found, and what it is currently investigating? How persistent, structured, and reliable is this internal model across the full duration of an engagement?

A summary comparison across all three generations—including reasoning capability, autonomy level, adaptability, and state-memory handling—is presented in Table 4.1 in Chapter 4, where the full dimensional analysis is synthesised after each generation has been examined in detail.

3.2 First Generation: Traditional Automated Scanning

3.2.1 Signature-Based Dynamic Testing

Nuclei, developed by ProjectDiscovery and publicly released in 2020, became the de facto community standard for signature-based dynamic scanning (ProjectDiscovery, 2020). Its architecture is straightforward in design: a YAML-based template engine encodes detection logic as a combination of a request specification (method, path, headers, body), a response matching rule (status code, header value, body substring or regex), and a severity classification. A library of over 7,000 community-maintained templates covers known CVEs, exposed administrative interfaces, default credentials, misconfigured cloud storage, and common version disclosure patterns.

The operational value of this approach is genuine and should not be dismissed. A Nuclei scan against a target with outdated, CVE-affected dependencies will reliably identify those dependencies with high precision and negligible false-positive rate, in a fraction of the time a manual review would require. For organisations seeking to ensure that their exposed software versions do not carry known, publicly exploitable weaknesses, signature-based scanning delivers measurable, reproducible value.

The structural analysis, however, exposes an inescapable boundary. Nuclei’s detection logic is entirely conditional on the prior existence of a template that encodes the specific vulnerability indicator the system is looking for. When Díaz-Verdejo et al. (2022) evaluated leading DAST tools against a corpus of intentionally vulnerable applications including custom business logic flaws, Nuclei’s recall rate on OWASP categories requiring semantic reasoning—Broken Access Control (A01), Insecure Design (A04), and Authentication Failures in their logic-flaw variant (A07)—was negligible (Díaz-Verdejo et al., 2022). This is not because Nuclei lacks templates for these categories; it is because *by design, no template can encode the semantic condition for detecting them*. Broken access control is not a fixed string in a response. It is not a specific HTTP status code. It is a relationship between what the user requested, who the user is, and what they are permitted to receive—a three-way relational condition that varies per application and cannot be universally pre-specified.

The maintenance dimension compounds the problem further. The CVE database adds tens of thousands of entries annually. Nuclei template coverage of newly disclosed vulnerabilities lags disclosure by weeks to months even with an active community. The gap between known vulnerabilities and template-covered vulnerabilities is structurally growing, not shrinking.

Evaluated across the four dimensions: Reasoning capability is zero—Nuclei cannot

infer; it can only recognise. Autonomy is moderate: the tool executes without intervention, but all behaviour is pre-specified. Adaptability is zero: the template set is static and does not respond to evidence gathered during the session. State-memory handling is minimal: Nuclei maintains a result log but has no model of its own investigation that would allow it to reason about what it has learned about the target.

3.2.2 Heuristic Active Scanning

OWASP ZAP (OWASP Foundation, 2010) represents a different architectural choice that partially addresses Nuclei’s complete template dependence. Rather than matching against specific known-vulnerability patterns, ZAP implements an active scanning engine that applies broad payload families across all discovered input surfaces and analyses responses for anomalous patterns. SQL meta-characters, path traversal sequences, XSS reflection tests, and header injection strings are systematically submitted to every discovered form field, URL parameter, and cookie value, and ZAP’s heuristic engine analyses the responses for deviations that suggest vulnerability.

The theoretical advantage of this approach is that it can, in principle, detect vulnerability instances for which no specific template exists—provided the vulnerability manifests as a response anomaly that ZAP’s heuristics can recognise. A custom SQL-injectable parameter in a bespoke application might be detected through a distinctive error message or response timing anomaly even without a specific template for that application’s database backend.

The practical limitation is twofold and well-documented. First, the false-positive rate is structurally high. Because ZAP applies injection payloads uniformly without any model of each parameter’s intended purpose, it generates anomalous responses from non-vulnerable parameters just as readily as from vulnerable ones. A search field that rejects malformed queries, a date picker that validates its input format, a text field with a maximum-length constraint—all of these produce responses that differ from what ZAP sends as input, and all of these differences look like vulnerability indicators to a context-blind heuristic engine. Díaz-Verdejo et al. found false-positive rates of 30 to 60% across evaluated applications (Díaz-Verdejo et al., 2022), meaning that a practitioner using ZAP must manually review, investigate, and dismiss the majority of its output before any actionable findings emerge. This manual review burden does not scale.

Second, ZAP shares the flat-finding limitation with all first-generation tools: it has no model of how its findings relate to each other. Each candidate finding is independently identified and reported. The practitioner must manually reason about whether the confirmed SSRF and the confirmed misconfigured internal service are connected, and whether that connection creates an exploitation path more severe than either finding in isolation. This cross-finding reasoning is the cognitive work that first-generation tools

entirely offload to the human.

Nessus and **OpenVAS** occupy a structurally similar position to Nuclei in the infrastructure domain: deep template libraries for CVE-based detection, high precision on known vulnerabilities, and near-zero coverage for anything that requires reasoning about application intent. They provide genuine value for compliance-oriented scanning and infrastructure hardening; they are not penetration testing tools in the full professional sense.

3.2.3 Domain-Specific Expert Tools: SQLMap as a Case Study

SQLMap (Stampar, 2006) illustrates a third approach worth examining: rather than broad coverage with limited depth, it achieves near-total coverage within a precisely delimited domain. SQLMap implements six distinct SQL injection detection techniques, sophisticated WAF fingerprinting and bypass logic, and post-exploitation capabilities including automated extraction of database schemas and contents. Within its domain, it is among the most reliable and thorough automated tools available, frequently outperforming junior human testers in the specific task of confirming and exploiting SQL injection.

The analytical significance of SQLMap is what its design reveals about the scalability of deep specialisation. SQLMap requires that a human practitioner identify SQL-injectable endpoints before the tool can be applied; it cannot perform the reconnaissance phase autonomously. It cannot reason about what a confirmed SQL injection enables in the context of the broader application. If the database user has file-write privileges that would allow writing a web shell, SQLMap will not autonomously connect these two facts and pursue the resulting Remote Code Execution opportunity unless explicitly configured to attempt it. The tool's depth within its domain is achieved by sacrificing breadth and the inter-domain reasoning that real engagements require.

This pattern—deep competence within narrow boundaries, fundamental incapacity for the cross-domain reasoning that connects findings into chains—is a defining characteristic of the first generation as a whole.

3.2.4 Summary: Why the Template Ceiling Cannot Be Crossed from Within

The common thread across all first-generation tools is the recognition paradigm: they can identify what they have been taught to look for, and nothing else. This is not an observation about implementation quality or community effort. It is a logical consequence of the paradigm. Any detection system that operates by pattern matching against a known-vulnerability model is bounded by the completeness of that model. Novel vulnerabilities, context-specific business logic flaws, and multi-step exploitation chains

that cross vulnerability categories are definitionally invisible to recognition-based systems.

The Template Ceiling. The template ceiling is not a resource problem. No amount of additional template-writing, engineering effort, or community contribution can enable a recognition-based system to detect a vulnerability class whose detection requires understanding what the application *is supposed to do*. This understanding is semantic, contextual, and application-specific—it is the foundation of expert penetration testing and cannot be encoded in a finite set of patterns.

3.3 Second Generation: Rule-Based Automation and Expert Systems

The second generation of automated security tools attempted to address the coordination and sequencing limitations of standalone first-generation scanners without abandoning the deterministic, rule-based foundation that makes their behaviour predictable and auditable. Orchestration frameworks and integrated exploitation suites—including the automation layers of platforms such as Metasploit (Kennedy et al., 2011)—applied conditional branching logic to chain tool invocations: a confirmed open port triggers a protocol-specific enumeration module, which may trigger a vulnerability-specific exploitation module upon confirming a matching version.

This advance is real but limited. Conditional branching introduces a primitive form of adaptability: the tool sequence is not fully fixed in advance but depends on what previous steps reveal. This is strictly better than the completely static behaviour of first-generation tools. However, the adaptability is bounded by the *coverage completeness assumption*: the system can only branch along paths that a human developer anticipated, defined, and programmed. Any environmental deviation from the anticipated conditions—an unexpected middleware layer, a non-standard authentication scheme, a combination of findings that creates a novel attack path not represented in the ruleset—leaves the system without a branch to follow. The ruleset exhausts itself, and the system stops adapting.

The second generation also brought real improvements in finding management: integrated tools maintain structured result databases that persist across the session and are queryable by later modules. This is an improvement over the flat result logs of first-generation tools. But the database is used for condition evaluation in subsequent rules, not for strategic inference. The system cannot ask “given all my confirmed findings taken together, what is the most valuable next step?” That question requires reasoning, not rule lookup.

The second-generation limitation, in summary, is the same as the first generation’s: the absence of a mechanism to reason about novel situations. The ruleset can be arbitrarily

large; it cannot be infinite. And real-world targets, in all their specific, contextual, business-logic-laden complexity, routinely present situations outside any finite ruleset.

3.4 Third Generation: AI-Driven Autonomous Agents

The emergence of frontier Large Language Models created the conditions for a qualitatively different approach. Rather than encoding security knowledge as templates or rules—both of which are finite and require prior specification—an LLM encodes security knowledge as a generalised representation that can be applied to novel instances. The agent does not need a template for the specific vulnerability in front of it; it needs to understand security well enough to reason about what the instance means.

3.4.1 Demonstrated Capabilities: Establishing the Baseline

CVE-Level Exploitation. Liu et al. (2024) demonstrated that a frontier LLM provided with structured CVE descriptions achieved 87% autonomous exploitation success; the same model without structured context achieved 7%—a 12.4-fold gap attributable solely to architectural scaffolding (Liu et al., 2024).

Hands-On Penetration Testing. Deng et al. (2023) showed PentestGPT completing ~67% of HackTheBox challenges versus 14.3% for an unaugmented baseline and ~40% for beginner human practitioners (Deng et al., 2023).

General Host Security Reasoning. Happe and Cito (2023) showed a minimally scaffolded LLM agent autonomously escalating privileges across target Linux hosts by reasoning from first-principles security knowledge (Happe and Cito, 2023).

These results establish that LLMs have crossed the template ceiling. The question is why these capabilities fail to produce reliable sustained performance across full engagements.

3.4.2 LLM-Based Agent Reasoning Paradigms: A Deep Analysis

The way an LLM’s reasoning is structured and elicited by the surrounding agent architecture is not a secondary concern. It is the primary determinant of whether the model’s intrinsic capability can be sustained across a multi-hour, multi-hundred-action security investigation. This section analyses the principal reasoning paradigms deployed in LLM-based security agents, examining each for both its demonstrated capabilities and its structural limitations in sustained penetration testing contexts.

The ReAct Paradigm: Interleaved Reasoning and Action

The Reason-Act (ReAct) paradigm, introduced by Yao et al. (2022), is the architectural foundation of virtually every LLM-based autonomous agent currently in the literature,

including all evaluated security agents (Yao et al., 2022). Its central design principle is the interleaving of explicit textual reasoning traces and tool invocations within a single continuous generation loop.

At each step, the agent receives the full accumulated conversation history as input. It first generates a textual reasoning trace—sometimes called a “thought”—that articulates its interpretation of the current evidence and what action it intends to take and why. It then generates an action specification: which tool to call and with what parameters. The tool is executed, its output is appended to the conversation as an observation, and the cycle repeats. The agent’s strategy evolves step by step, with each reasoning trace shaped by all preceding observations.

Why ReAct advances the state of the art: ReAct’s interleaving design enables a form of real-time hypothesis revision that neither pure planning (which commits to a plan before taking any action) nor pure reaction (which takes actions without explicit reasoning) provides. When an observation at step t contradicts the expectation that motivated the action at step $t - 1$, the reasoning trace at step $t + 1$ can explicitly acknowledge the contradiction and revise the strategy. Yao et al. demonstrated that this mid-task course correction produces substantially better performance than chain-of-thought reasoning alone or action execution alone on multi-step tasks requiring information gathering (Yao et al., 2022). For penetration testing—where almost every probe returns unexpected information that should update the tester’s model of the target—this adaptability is theoretically well-matched to the task structure.

The structural failure of ReAct in sustained security contexts: The fundamental problem with ReAct as applied to penetration testing is that the agent’s *sole persistent memory of its entire session* is the conversation history accumulating in its context window. Every reasoning trace, every action specification, every tool observation is appended to this linear text buffer. As the session extends, the buffer grows. After several hundred iterations of a real engagement—a modest session for a complex web application—the buffer may contain tens of thousands of tokens of HTTP responses, scan output, and error messages.

Three distinct failure modes emerge from this design:

First, *context window overflow*: when the accumulated history exceeds the model’s context limit, some portion of early history must be truncated or compressed. Any compression operation is inherently lossy. Confirmed findings from early in the session, rejected hypotheses that should exclude certain approaches from further consideration, and strategic reasoning about multi-step chains—all of these are at risk of being absent or distorted at the point in the session when they are most needed.

Second, *attention dilution within the window*: Liu et al. (2023) documented that even information that technically remains within the context window is not recalled with uniform

reliability (Liu et al., 2023). The transformer’s self-attention mechanism exhibits primacy and recency bias; information positioned in the middle of a long context is under-attended relative to what precedes and follows it. A confirmed vulnerability finding from 80 steps ago may be literally present in the context but effectively inaccessible to the model’s generation at step 160—not because it was truncated but because the attention mechanism has moved on.

Third, *action space inconsistency*: because the agent has no structured record of what it has tested, it cannot determine from conversation history alone whether it has already tried a specific payload against a specific endpoint. The history *contains* this information, but it is embedded in natural language that may be phrased differently each time the model references it, and it may be diluted by hundreds of subsequent actions. The result is that the agent will regularly re-test surfaces it has already exhausted—the “thrashing” behaviour documented across empirical evaluations of PentestGPT and HackingBuddyGPT—not because it has made a strategic decision to retry, but because it has lost reliable access to the record of its own prior actions.

ReAct in Security: Capability and Structural Limit. ReAct enables genuine real-time hypothesis revision grounded in observed evidence—a qualitative advance over both pure planning and pure pattern matching. Its structural limit is that the trajectory-as-memory model scales linearly with session length while the context window is fixed, making sustained coherence across real-world engagement durations structurally unstable. This is an architectural problem with the ReAct paradigm as applied to long-horizon tasks, not an LLM capability problem.

Chain-of-Thought: Deliberative Planning and Its Failures

Chain-of-Thought (CoT) prompting (Wei et al., 2022) elicits explicit multi-step verbal reasoning before any action is taken—a planning paradigm. The security reasoning advantage of CoT is measurable on isolated, closed-world tasks: explicit step-by-step articulation prevents premature conclusions and produces outputs more easily scrutinised for logical errors.

The limitation is *open-world planning failure*. CoT planning assumes the world state at planning time persists through execution. In penetration testing this is systematically violated: a WAF that blocks certain payloads, a rate limiter that collapses timing-based inference, or an authentication mechanism that behaves differently to automated requests can all invalidate a CoT plan at its first execution step. Hybrid CoT-ReAct approaches introduce *plan-reality divergence*: when early execution contradicts the CoT plan, the agent must either follow an inapplicable plan or abandon the computation. Neither resolution is principled.

Tree of Thoughts: Exploratory Deliberation

Tree of Thoughts (ToT) (Yao et al., 2023) extends CoT to a tree-structured exploration of alternative reasoning paths: generating multiple candidate next-steps, evaluating their promise, and selecting the most promising branch. For security agents, this enables holding multiple competing exploitation hypotheses simultaneously without premature commitment.

The practical limitations are severe for sustained engagements: tree expansion requires at least two LLM calls per branch (generation and evaluation), costs multiply at each tree level, and the utility evaluator suffers the same long-context degradation as the primary reasoning model. Published ToT applications to security tasks have been limited to short-horizon problems; extension to full-engagement security assessment remains an open theoretical problem.

Tool-Augmented Inference: Capabilities and Their Costs

Tool augmentation—equipping an LLM agent to invoke external functions and receive their outputs—elevates security reasoning from speculation to grounded analysis. Schick et al. (2023) demonstrated that LLMs can learn to determine when and how to use tools through exposure to productive and unproductive invocation examples (Schick et al., 2023); Qin et al. (2023) extended this to generalised tool-use across tool sets not seen in fine-tuning data (Qin et al., 2023). Despite these capabilities, tool augmentation introduces three structural costs:

Context pollution by tool output: Raw HTTP responses and verbose scan results can consume thousands of tokens per invocation, accelerating trajectory explosion and displacing strategic reasoning traces.

Selection noise in large tool spaces: Without a structured model of what has already been tried, the agent’s tool selection may not reflect accumulated evidence of what has and has not been productive.

Output parsing reliability: Translating unstructured, format-variable tool output into structured inferences requires LLM generation that is subject to hallucination (Ji et al., 2023). Incorrect inferences committed to the trajectory shape all subsequent reasoning without a mechanism for correction.

The Planning Versus Reactive Execution Tension

The tension between deliberative planning and reactive execution is one of the oldest problems in AI agent design (Russell and Norvig, 2020). Deliberative planning provides strategic coherence but is brittle to unexpected evidence. Reactive execution adapts fluidly

but lacks strategic direction. ReAct occupies an unstable middle ground: verbal reasoning traces provide local strategic guidance, but that guidance degrades as context compresses. No architecture in the published security agent literature has resolved this tension in a session-length-independent way; it remains an open research problem.

3.4.3 Memory Mechanisms in LLM-Based Security Agents

Memory is the central architectural property determining whether a system can sustain coherent, non-redundant strategy across an extended engagement. Three approaches have been applied in LLM-based systems:

In-context trajectory memory (universal baseline): the entire session history is accumulated in the context window. Simple and complete, but subject to context overflow, attention dilution, and the complete absence of structured query capability. An agent cannot efficiently answer “have I tested IDOR on this endpoint with role elevation?” without scanning the entire natural-language history.

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020): an external vector store holds information; semantically relevant subsets are retrieved per step. Solves overflow, but security agents need *structured* queries (“what is the current status of hypothesis H7?”), not semantic similarity retrieval. RAG cannot distinguish “I tested this endpoint” from “I hypothesised about testing this endpoint,” and cannot reflect state mutations as a hypothesis transitions from open to confirmed.

External structured memory (MemGPT) (Packer et al., 2023): the agent explicitly manages reads and writes to a structured external store. Architecturally closest to what penetration testing requires. The structural limitation is the *agent interface problem*: reads and writes are themselves LLM-generated; a hallucinated status update is committed to the persistent store and corrupts all subsequent reasoning with no independent verification mechanism.

3.4.4 Critical Analysis of Deployed LLM Security Agents

PentestGPT

PentestGPT (Deng et al., 2023) separates the monolithic agent into three specialised modules: a *reasoning module* for high-level strategic decisions, a *generation module* for translating decisions into tool commands, and a *parsing module* for interpreting tool outputs. The performance impact is dramatic: the same underlying model achieves 14.3% task completion on HackTheBox challenges without the module structure; with it, this rises to 67%. Architecture, not the model, accounts for the majority of the gain—a direct empirical validation of H1.

The failure analysis is equally instructive. Failed tasks disproportionately concentrate

on challenges requiring many sequential steps, not on those with the highest technical difficulty (Deng et al., 2023). This is the empirical signature of the trajectory explosion problem: the architecture successfully separates *types* of reasoning but does not solve the *duration* problem.

HackingBuddyGPT

HackingBuddyGPT (Happe and Cito, 2023) provides a minimalist contrast: a frontier LLM with standard shell access and a brief system prompt, no domain-specific scaffolding. Its performance demonstrates that LLM security reasoning is generalisable without elaborate architecture. The failure modes are analytically clear: the most common pattern is command repetition—the agent reissues commands already issued because it cannot reliably determine from conversation history what it has already tried. The second most common is strategic incoherence after long sessions: the agent loses the thread of an established escalation path and issues unrelated commands. Both are direct manifestations of the action deduplication gap and attention dilution problem.

AutoAttacker

AutoAttacker (Xu et al., 2024) is the only system in the surveyed literature that explicitly addresses the memory problem architecturally. Its periodic heuristic summarisation step compresses accumulated conversation history and replaces raw history in the context window. This directly addresses context overflow. The limitation is that it treats the symptom rather than the cause: summarisation loses the structured facts subsequent reasoning requires. A summary stating “SQL injection attempts on the product search endpoint with mixed results” loses which specific payloads were tried, what the precise HTTP responses were, and what follow-up probes those patterns suggest—the structured data from which the next steps must be derived. AutoAttacker improves scalability at the cost of fact preservation.

Table 3.1: LLM-Based Security Agents: Comparative Dimensional Analysis

System	Reasoning Archi- tecture	Autonomy	Adaptability Con- straint	Memory Strat- egy and Limita- tion
PentestGPT (Deng et al., 2023)	Module-separated (Reasoning / Generation / Parsing)	Semi-autonomous human confirmation at checkpoints	Evidence-driven within each module; no cross-module persistent state	Disjoint per-module trajectories; confirmed findings not durably tracked across modules
HackingBuddyGPT (Happe and Cito, 2023)	Monolithic reactive loop; minimal scaffolding	Fully autonomous over session	Directly responsive to terminal output; no global strategy model	Uncompressed linear conversation history; loses coherence after ~100 steps
AutoAttacker (Xu et al., 2024)	CoT planning phase followed by re-active execution loop	Fully autonomous	Moderate; plan-reality divergence accepted; reactive phase adapts locally	Heuristic natural language summarization; compresses away specific structured facts

3.4.5 Cross-Cutting Analysis: What the Convergence of Failures Reveals

The most analytically significant observation across the three evaluated systems is the convergence of their primary failure modes despite their architectural diversity. PentestGPT uses module separation and careful prompt design. HackingBuddyGPT uses minimal scaffolding and raw model capability. AutoAttacker uses heuristic summarisation and planning phases. These are substantively different architectural approaches. Yet all three fail predominantly on tasks requiring sustained, structured awareness of what has been done and found over the course of a long session.

This convergence is not coincidental. It is a logical consequence of the one property all three systems share: the trajectory-as-memory model. In all three, the agent’s primary persistent state is stored in natural language conversation history. The architectural differences between the systems are differences in how that history is managed (compressed, modularised, or raw), not differences in its fundamental nature. As long as the trajectory remains the primary memory substrate, the consequences of its structural limitations—context overflow, attention dilution, absence of structured queries—will manifest in essentially all systems that share it, regardless of how the surface-level architecture differs.

Chapter 4 Synthesis: Causal Mechanisms of Convergent Failure

The comparative analysis in Chapter 3 documented a striking empirical pattern: architecturally diverse LLM-based security agents—PentestGPT with module separation (Deng et al., 2023), HackingBuddyGPT with minimal scaffolding (Happe and Cito, 2023), AutoAttacker with heuristic summarisation (Xu et al., 2024)—converge on the same primary failure modes. This chapter synthesises those observations into a causal account that explains not merely *that* these failures occur, but *why* they must occur in any system sharing the trajectory-as-memory architectural foundation.

4.1 Comprehensive Cross-Generational Comparison

Before articulating causal mechanisms, the full dimensional comparison across all three generations consolidates the empirical basis from which those mechanisms are derived.

Table 4.1: Comprehensive Cross-Generational Comparison of Automated Penetration Testing Approaches

Dimension	Traditional Tools (Gen 1)	Rule-Based Systems (Gen 2)	AI-Driven Agents (Gen 3)
Reasoning Capability	None: pattern matching against pre-encoded signatures only	None: conditional branching within pre-programmed rulesets	High within short sessions: semantic inference from observed evidence; degrades with session length
Autonomy Level	Moderate: executes without intervention but all behaviour is pre-specified	Moderate-high: conditional branching adapts execution order	High: self-directed action selection from open-ended strategy space
Adaptability	None: static template or payload set applied uniformly	Rule-based only: adapts along pre-anticipated branches	Evidence-driven: forms new hypotheses from unexpected observations; adaptation quality decays as context degrades
State/Memory Handling	Flat result logs with no relational structure	Structured finding databases queryable by subsequent rules	Volatile conversation trajectory subject to overflow, dilution, and compression loss
Primary Failure Modes	Template ceiling: zero coverage of vulnerabilities requiring semantic reasoning Díaz-Verdejo et al. (2022); no cross-finding chain reasoning	Coverage completeness failure: cannot respond to unanticipated environmental conditions; no reasoning beyond encoded rules Kennedy et al. (2011)	Context degradation: strategic incoherence after ~100 steps Happe and Cito (2023); action repetition from lost history Deng et al. (2023); hallucinated 24 confirmations from unverified

Table 4.1 reveals a structural progression: each generation overcomes the prior generation’s binding constraint while introducing a qualitatively different one. Gen 1 tools are constrained by the template ceiling—they cannot reason beyond pre-encoded patterns. Gen 2 tools are constrained by the coverage completeness assumption—they cannot adapt beyond pre-anticipated conditions. Gen 3 agents are constrained by the trajectory-as-memory model—they cannot sustain the reasoning they demonstrably possess across engagement-length sessions. The progression is from a capability deficit to a sustainability deficit: the fundamental problem shifts from *whether* the system can reason to *how long* it can maintain coherent reasoning.

4.2 Causal Mechanism 1: The Information Bottleneck

The context window is a hard constraint on the total information available to an LLM at any generation step. The transformer architecture (Vaswani et al., 2017) processes a fixed-length token sequence; information outside this sequence does not exist for the model. As a penetration testing session progresses, the volume of accumulated observations—HTTP responses, scan outputs, error messages, prior reasoning traces—grows monotonically. The context window capacity does not grow. The ratio of available-to-required information therefore decreases monotonically with session length.

This creates a forced trade-off with no satisfactory resolution within the trajectory-as-memory model. When accumulated content exceeds window capacity, the system must choose among three lossy strategies: truncation (discarding early content entirely), compression (summarising content into shorter representations), or selective retention (keeping only content deemed most relevant). Each strategy produces a distinct failure mode. Truncation directly causes the loss of early-session findings—a vulnerability confirmed at step 20 that is truncated at step 120 cannot inform exploitation reasoning at step 150. AutoAttacker’s heuristic summarisation (Xu et al., 2024) is an explicit acknowledgment of this problem, but as the analysis in Section 3.4 established, summarisation trades overflow for specificity loss, converting structured facts into vague natural-language approximations.

The “lost in the middle” effect (Liu et al., 2023) compounds the bottleneck within the window itself: even information that has not been truncated or compressed is not accessed uniformly. Content in the middle of a long context receives systematically less attention than content at the beginning or end. For a security agent whose most strategically important findings may be located at arbitrary positions in the conversation history, this attention non-uniformity is functionally equivalent to partial information loss—not because the data was removed, but because the self-attention mechanism under-weights it during generation.

The information bottleneck is not resolved by increasing context window size. Larger

windows delay the onset of truncation but do not eliminate it; and attention dilution effects worsen, not improve, as window size increases (Liu et al., 2023). The bottleneck is a structural consequence of storing unbounded session state in a bounded-capacity medium, and it cannot be eliminated by expanding the medium.

4.3 Causal Mechanism 2: Implicit State Representation

The second mechanism is the absence of explicit, structured state external to the conversation history. In the trajectory-as-memory model, every fact about the investigation—which endpoints have been tested, which hypotheses are confirmed, which payloads have been tried against which parameters—is encoded implicitly in natural language within the conversation trajectory. There is no structured representation that can be queried independently of the full conversation content.

This implicit encoding produces three consequences that directly generate the observed failure patterns.

First, there is no deduplication mechanism: the agent cannot efficiently determine whether a specific action has already been taken without reading through the entire conversation history, which may itself be truncated or attention-diluted. The command repetition documented in HackingBuddyGPT (Happe and Cito, 2023)—agents re-issuing commands they have already executed and received results for—is the direct empirical consequence of this absence.

Second, there is no structured relationship representation between findings. An expert human tester maintains a mental model (or explicit notes) of which findings relate to which others and what exploitation chains they collectively enable. In the trajectory-as-memory model, these relationships exist only as natural-language assertions scattered across the conversation history. Retrieving the full set of relationships involving a specific finding requires locating and correctly interpreting every relevant assertion—a task whose reliability degrades as the conversation grows and attention dilutes. The failure to pursue confirmed exploitation chains documented in PentestGPT’s long-session evaluations (Deng et al., 2023) is a direct manifestation of this representational deficiency.

Third, the implicit representation cannot distinguish between facts of different epistemic status. A hypothesis confirmed through verified exploitation, a hypothesis speculatively mentioned in a reasoning trace but never tested, and a hypothesis tested and definitively rejected all appear as natural language in the same conversation stream. Without explicit status markers maintained in a structured store, the agent’s ability to distinguish between these states degrades as the conversation accumulates more content with mixed epistemic

status.

The combination of these three consequences—no deduplication, no relational structure, no epistemic status tracking—constitutes the causal mechanism that produces the thrashing and strategic incoherence failure patterns documented across all three evaluated systems (Deng et al., 2023; Happe and Cito, 2023; Xu et al., 2024).

4.4 Causal Mechanism 3: Unverified Inference Propagation

The third mechanism concerns the reliability of the agent’s own generated assertions. When an LLM-based agent interprets a tool output and generates a claim about target state—“this parameter is vulnerable to SQL injection,” “this endpoint enforces access control correctly”—that claim becomes part of the conversation trajectory and shapes all subsequent reasoning. If the claim is incorrect, the error propagates: subsequent actions are chosen on the basis of a false premise, and their results are interpreted through the lens of that false premise, compounding rather than correcting the original error.

Ji et al.’s survey of hallucination in natural language generation (Ji et al., 2023) establishes that LLMs generate plausible but factually incorrect content at non-negligible rates, with hallucination frequency increasing under conditions of uncertainty and long-context generation—precisely the conditions that prevail in extended security assessments. In a security context, the consequences of hallucinated findings are particularly acute: a falsely confirmed vulnerability causes the agent to spend subsequent actions attempting to exploit a non-existent weakness, while a falsely rejected hypothesis causes a genuine vulnerability to be permanently excluded from investigation.

The critical architectural observation is that in every current system, the model that generates the claim is the same model that subsequently acts on it, with no independent verification layer between generation and commitment to the persistent record. When the model at step t asserts “injection confirmed,” that assertion is accepted into the trajectory as fact and informs reasoning at steps $t+1, t+2, \dots$ without any mechanism to verify the assertion against actual tool output evidence. This is an architectural decision to trust unverified model output—appropriate for short-horizon tasks where errors are quickly observable, but structurally inappropriate for sustained operations where errors compound across hundreds of subsequent steps.

4.5 Compound Degradation: How the Three Mechanisms Interact

The three mechanisms are not independent; they interact to produce failure patterns more severe than any individual mechanism would cause in isolation.

The information bottleneck (Mechanism 1) degrades the context available for reasoning, which increases the probability of incorrect inferences. Those incorrect inferences (Mechanism 3) enter the trajectory as implicit assertions (Mechanism 2) that cannot be distinguished from verified facts. When the context is subsequently compressed, the compression operation cannot selectively preserve verified facts over hallucinated ones because the implicit state representation does not distinguish between them. The compressed context therefore inherits errors from prior reasoning, and subsequent generation operates from a context that is both incomplete (due to the bottleneck) and partially incorrect (due to unverified propagation).

This compound degradation explains the monotonic relationship between session length and failure rate observed across all evaluated systems (Deng et al., 2023; Happe and Cito, 2023). The degradation is not random; it is directional. Each compression event loses information and potentially preserves errors; each subsequent generation reasons from a progressively less reliable context. The system does not merely fail to improve with session length—it actively degrades, accumulating the consequences of its own earlier reasoning errors in a context that becomes progressively less capable of detecting or correcting them.

The implication for architectural design is that addressing any single mechanism in isolation yields limited benefit. Expanding the context window (Mechanism 1) without introducing structured state (Mechanism 2) or verification (Mechanism 3) delays degradation but does not prevent it. Introducing structured state without verification produces a more organised record that still contains unverified assertions. Only a design that addresses all three mechanisms concurrently—externalising state to a structured store, verifying assertions before committing them to that store, and decoupling state management from the bounded context window—can break the compound degradation cycle. This conclusion directly informs the research gap taxonomy presented in the following chapter.

Chapter 5 Research Gap Taxonomy

The comparative analysis in Chapter 3 and the causal synthesis in Chapter 4 jointly establish the evidential basis for a formal taxonomy of the field’s open research problems. Six gaps are identified. Each is stated clearly, justified by specific evidence from the analysis, and characterised in terms of why it cannot be closed by optimisation within existing paradigms.

5.1 Gap Taxonomy

G1 – Semantic Generalisation Deficit. No existing automated penetration testing system can reliably detect vulnerability classes whose identification requires reasoning about application intent—including business logic flaws, contextual access control failures, and semantically novel instances of known vulnerability types—without prior encoding of the specific vulnerability’s signature.

Evidence: Díaz-Verdejo et al.’s evaluation (Díaz-Verdejo et al., 2022) confirmed near-zero recall on OWASP A01 (Broken Access Control) and A04 (Insecure Design) across all evaluated DAST tools. These categories represent the most prevalent real-world vulnerability classes and are structurally inaccessible to recognition-based tools by the analysis in Section 3.2. LLM-based agents partially address this gap—PentestGPT’s 67% success rate on semantically complex HackTheBox challenges establishes partial closure—but do not fully close it, as evidenced by the remaining 33% failure rate and the documented degradation on long-session tasks where maintained semantic context is required.

Why the gap persists: Full closure requires both the semantic reasoning capability that LLMs provide and the sustained, reliable access to structured contextual information that current architectures do not provide. The gap is compound in nature: addressing the reasoning component (the model can reason) without addressing the scaffolding component (the model cannot access its accumulated evidence reliably at scale) produces the partial performance demonstrated by current systems, not the full performance that expert human testers achieve.

G2 – Heuristic Brittleness and False Positive Inflation. Context-blind payload injection produces false positive rates of 30–60% on complex applications, requiring manual triage that negates the scalability benefit of automation and systematically erodes practitioner trust in automated findings.

Evidence: Díaz-Verdejo et al.’s evaluation reported false-positive rates in this range across evaluated DAST tools (Díaz-Verdejo et al., 2022). The causal mechanism is context blindness: uniform payload application across all parameters treats non-vulnerable input validation as indistinguishable from vulnerable processing—a consequence of having no model of each parameter’s intended purpose.

Why the gap persists: The false-positive problem is not addressable by refining payloads, because the distinction between a vulnerable and a non-vulnerable parameter is not observable from the parameter’s response to injection alone; it requires understanding the parameter’s intended purpose, which requires semantic reasoning about the application. LLM-based agents can, in principle, apply this semantic reasoning to winnow false positives; but the extent to which current LLM agents actually reduce false-positive rates relative to traditional DAST tools has not been systematically measured in the published literature.

G3 – Finding-Relationship Chain Blindness. No existing automated penetration testing system maintains a persistent, queryable representation of the relationships between confirmed findings that would enable systematic reasoning about multi-step vulnerability chains across the full duration of an engagement.

Evidence: LLM-based agents can perform chain reasoning within a single inference step—the model, given confirmation of an SSRF finding, can articulate what subsequent steps the finding enables. But this chain reasoning exists only in the moment of its generation. After compression, the chain context from 50 steps ago may be inaccessible at step 150, and the opportunity is missed. The failure to pursue confirmed exploitation chains, documented in PentestGPT’s long-session failures (Deng et al., 2023), is the empirical signature of this gap.

Why the gap persists: A persistent finding-relationship graph—one that survives context compression events and can be queried at any point in the session—is not implemented in any published security agent architecture. Its absence is a consequence of the trajectory-as-memory model, which stores all finding information in natural language conversation history rather than a queryable structured representation.

G4 – Stateless Reasoning and Context Degradation. Every LLM-based penetration testing system in the published literature relies on conversation trajectory as its primary persistent state. Confirmed findings, rejected hypotheses, and tested payloads are subject to irreversible loss through context compression and systematic under-attention, causing strategic degradation that worsens monotonically with session length.

Evidence: This gap is directly evidenced by the correlation between session length and failure rate in PentestGPT (Deng et al., 2023), the command-repetition failures in HackingBuddyGPT (Happe and Cito, 2023), and the explicit memory mitigation

strategy (heuristic summarisation) deployed by AutoAttacker (Xu et al., 2024). The causal mechanism is established in the synthesis: the information bottleneck of the context window is a fixed capacity meeting exponentially growing session content.

Why the gap persists: Addressing this gap requires externalising essential testing state to a store that is independent of the context window and immune to compression events. No current system achieves this for the specific structured state (hypothesis status, payload history, coverage tracking) that penetration testing requires. Larger context windows delay but do not eliminate the problem; heuristic summarisation trades overflow for fact loss.

G5 – Absent Hypothesis Lifecycle Management. No existing LLM-based security agent maintains a formal, structured registry of security hypotheses by lifecycle status (open, under investigation, confirmed, rejected) with associated payload deduplication, causing redundant retesting after compression events and preventing principled exclusion of exhausted surfaces from future action selection.

Evidence: The thrashing behaviour documented in both PentestGPT and Hacking-BuddyGPT evaluations—agents retesting surfaces they have already exhausted—is the direct empirical signature of this gap (Deng et al., 2023; Happe and Cito, 2023). Expert human testers manage their hypothesis space explicitly, either mentally or through notes; current automated agents do not.

Why the gap persists: Hypothesis lifecycle management requires a structured state representation in which the lifecycle status of each surface-vulnerability pair is maintained independently of the conversation history and is updatable reliably. This is the same architectural requirement as G4 (external structured state), applied specifically to the hypothesis management use case. The gap is therefore a special case of the broader stateless reasoning problem, but merits separate identification because its operational consequences (payload redundancy, failure to converge) are distinct from the general strategic incoherence caused by G4.

G6 – Safety and Accountability Architecture Absence. No existing LLM-based penetration testing system provides safety enforcement adequate for responsible autonomous offensive operation: no formal resistance to prompt injection via target-embedded content, no verifiable scope enforcement, and no cryptographically secured audit trail.

Evidence: Prompt injection via adversarially crafted server responses is a documented and demonstrated attack vector against LLM-integrated applications (Greshake et al., 2023). A security agent that processes target HTTP responses directly as part of its reasoning context is structurally exposed to this attack: a malicious target can embed instruction-like content in its responses that, if processed without sanitisation, may redirect the agent’s

planning toward actions outside its authorised scope. Current security agent systems address this risk through soft constraints communicated via system prompt conditioning—an approach that is definitionally bypassable by adversarial content specifically designed to override it.

Why the gap persists: Closing this gap requires safety enforcement mechanisms that are architecturally independent of the LLM’s reasoning layer: scope validators that check targets before network connections are established, prompt injection filters that sanitise tool outputs before they reach the context window, and audit logging maintained in a form the agent cannot modify. These are infrastructure concerns that the current literature treats as engineering add-ons rather than architectural requirements.

5.2 Gap Interactions and Compounding Effects

Table 5.1: Research Gap Taxonomy: Evidence Map and Interaction Matrix

Gap	Primary Evidence	Evicted	Paradigm Affected	Compounds With
G1 – Semantic Generalisation	Díaz-Verdejo et al., 2022; Liu et al., 2024)		All generations	G4
G2 – Heuristic Brittleness	Díaz-Verdejo (30–60% FPR) (Díaz-Verdejo et al., 2022)		Gen 1; partially Gen 3	G1
G3 – Chain Blindness	PentestGPT failure analysis (Deng et al., 2023)		Gen 3 only	G4, G5
G4 – Stateless Reasoning	All Gen 3 systems; Liu et al. (Liu et al., 2023)		Gen 3 only	G3, G5, G6
G5 – Hypothesis Lifecycle	Thrashing in PentestGPT, HackingBuddyGPT (Happe and Cito, 2023)		Gen 3 only	G4
G6 – Safety Absence	Greshake et al. (Greshake et al., 2023)		Gen 3 only	G4

The gaps are not independent. G3, G4, and G5 form a particularly destructive compounding cluster. An agent without persistent state (G4) cannot track hypothesis lifecycle status (G5), and without hypothesis lifecycle tracking, it cannot maintain the record of confirmed findings required to reason about exploitation chains (G3). The failure

to discover multi-step vulnerability chains in long-session evaluations is not attributable to any single gap in isolation; it is the combined effect of all three. A system that addresses G4 alone, without also addressing G5 and G3, will be better at maintaining session continuity but will still fail to converge on chain exploitation because it lacks the structured hypothesis and finding representations needed to reason about chains at scale.

Similarly, G6 is not independent of G4 and G5. An agent that does not reliably know what it has done (G4, G5) cannot produce a meaningful audit trail, because the agent's own record of its actions is unreliable. The accountability component of G6 depends on resolving the memory components of G4 and G5.

Chapter 6 Conceptual Perspective

The research gap taxonomy identifies what is missing. This chapter articulates the theoretical properties that future approaches must provide—not as architectural blueprints or engineering specifications, but as conceptual requirements derived by logical inversion of the identified gaps.

6.1 The External State Principle

The trajectory-as-memory model is the common root cause of G3, G4, and G5. Inverting this root cause yields a theoretical principle: *for any autonomous agent operating over a decision horizon that exceeds what a single context window can reliably represent, the decision-relevant state of the ongoing investigation must be maintained in a structured store that is independent of the language model’s context, immune to compression events, and accessible through structured queries.*

This principle implies a specific partition of responsibility between the LLM and the memory system. The LLM’s responsibility is reasoning: given a compact, structured digest of the current state of the investigation, produce the next reasoning step and action selection. The memory system’s responsibility is state: maintain the ground truth of what has been tested, what has been found, and what the current strategic priorities are—not as an inference from natural language history, but as a directly maintained structured record.

This is a different model from both pure in-context memory and from RAG. In pure in-context memory, the LLM is responsible for both state maintenance and reasoning, and the state is entangled with the reasoning in natural language history. In RAG (Lewis et al., 2020), retrieval is semantic rather than structured, and the retrieved information does not reflect mutable state. The External State Principle requires that the memory system explicitly manage state transitions—when a hypothesis is confirmed, the memory system records the transition; when an endpoint is tested with a specific payload, the memory system records the action—so that the LLM is never responsible for inferring state from history.

The consequence for system design directions is clear: priority should be placed not on context window size (which delays but does not eliminate the trajectory explosion problem) but on externalising the structured facts that penetration testing’s decision logic most critically depends on.

6.2 The Verified Evidence Principle

The unverified inference propagation mechanism identified in Chapter 4 suggests a second theoretical principle: *the confirmation of a security finding must require observable, independently verifiable exploitation evidence, not merely an LLM’s generated assertion of confirmation.*

This principle addresses the hallucination risk documented across LLM systems (Ji et al., 2023) by introducing an architectural demarcation: a hypothesis transitions from “under investigation” to “confirmed” only when a verification layer external to the LLM generation process has examined the evidence and determined that it meets a defined confirmation standard. For a SQL injection hypothesis, confirmation requires that specific data has been extracted from the database by a query that the agent constructed—not that the agent’s reasoning trace asserts that injection was successful. For an access control hypothesis, confirmation requires that a response was received containing data from another user’s account—not that the agent inferred this was probably the case.

This verification boundary serves two distinct purposes. First, it prevents hallucinated confirmations from propagating into subsequent strategic reasoning, because the external verifier catches false assertions before they are recorded as facts. Second, it provides the empirical basis for meaningful reporting: a finding reported through a verified-evidence architecture can be demonstrated to a stakeholder with the specific, reproducible exploitation evidence that confirmed it, not merely with a reasoning trace asserting its probability.

6.3 The Hierarchical Decomposition Principle

The planning-versus-reaction tension—a foundational problem in AI agent design (Russell and Norvig, 2020)—and the chain blindness of single-agent loops (G3), jointly suggest a third theoretical orientation: *the exploration-exploitation and breadth-depth trade-offs that single sequential agent loops cannot resolve optimally may be addressable through hierarchical agent decomposition with shared external state.*

In a hierarchical architecture, a controlling agent maintains strategic oversight—managing the external state representation, identifying the most valuable next investigation direction based on the current finding graph, and delegating specific investigation tasks to specialised sub-agents. Each sub-agent operates with a constrained, well-defined context (a specific endpoint, a specific vulnerability class, a specific exploitation chain) and terminates when its task is complete, returning its findings to the external state for the controlling agent to incorporate. Because sub-agents operate concurrently, the exploration-exploitation tension is resolved by allocation rather than scheduling: one sub-agent explores new surfaces while another exploits a confirmed finding.

The critical enabling condition for this architecture is the external state store required by the External State Principle: without a shared, persistent, structured record of findings, concurrent sub-agents would either duplicate effort or miss interaction effects between their respective investigations. The hierarchy resolves the strategic coordination problem; the external state enables the hierarchy to function without constant full-context communication between agents.

Chapter 7 Discussion

7.1 Implications for Cybersecurity Practice and Governance

Automation complacency. Organisations relying on automated scanning as their primary assessment mechanism risk a structural false sense of coverage. First-generation tools cannot detect the most prevalent vulnerability classes (A01, A04) without semantic reasoning, and LLM-based agents degrade silently in ways invisible to volume-based monitoring. Governance frameworks must explicitly account for the Semantic Generalisation Deficit (G1) and False Positive Inflation (G2) when interpreting automated tool outputs.

LLM over-optimism. Liu et al.’s 87% CVE exploitation rate (Liu et al., 2024) reflects capability on a constrained task—exploiting vulnerabilities given structured descriptions—not general penetration testing. The same study’s 7% without structured context is the more relevant baseline for organisations deploying LLM-based systems without architectural scaffolding. Practitioners must understand the distinction between demonstrated short-horizon capability and undemonstrated sustained engagement performance.

Human-AI collaboration. The analysis supports a nuanced positioning: LLM-based agents are genuinely capable contributors to reconnaissance and initial surface-coverage phases, where their semantic generalisation over known patterns provides measurable value. The phases where they currently fail—sustained multi-step chain exploitation, long-horizon coherence, reliable finding confirmation—are precisely where expert human judgment is most valuable. A collaborative model, in which agents provide coverage and humans execute deep-reasoning exploitation, is both more scalable than purely human assessment and more reliable than fully autonomous AI.

7.2 Limitations of This Analysis

Benchmark heterogeneity. The systems analysed were evaluated on different targets, session budgets, and LLM backends—PentestGPT on HackTheBox, HackingBuddyGPT on custom Linux hosts, AutoAttacker on an undisclosed web application set. Direct quantitative comparison is not methodologically valid; comparative claims in this thesis are qualitative and structural.

Laboratory target bias. All published results involve intentionally vulnerable applications or purpose-built hosts. Production environments differ in WAF protection,

rate limiting, real-user traffic noise, and proprietary authentication mechanisms. The performance gap between laboratory and production settings is unknown and likely substantial.

Model evolution. The empirical results reflect models from 2022–2024. Current models are substantially more capable on many benchmarks. However, the structural limitations identified—context window finiteness, attention non-uniformity, absent external state—are architectural properties independent of model capability, making the Architecture Primacy argument durable.

7.3 Open Challenges and Research Risks

Benchmark imperative. The absence of a standardised penetration testing benchmark is the field’s most consequential methodological gap. The G1–G6 taxonomy provides a natural framework: a target corpus with at least one verified instance per gap, evaluated under reproducible conditions with defined iteration budgets and ground-truth confirmation criteria.

Dual-use tension. Every architectural improvement enabling sustained exploitation also enables autonomous attack without authorisation. Responsible disclosure norms, access control frameworks, and deployment governance must be first-class research concerns, not afterthoughts.

Formal verification. Safety properties—verifiable scope enforcement, prompt injection resistance, cryptographically auditable action records—require formal verification, not empirical testing alone. A system that passes a large adversarial test set remains exposed to unanticipated attacks. Applying model checking and verified isolation to autonomous security agents is an open problem the current literature has not substantively addressed.

Chapter 8 Conclusion

8.1 Summary of Findings

This thesis has conducted a rigorous critical analysis of automated penetration testing across three generations, directed toward two goals: identifying the precise structural causes of the performance gap between automated systems and expert human assessment, and establishing a justified taxonomy of the specific research problems that must be solved to close that gap.

The first generation demonstrated that automation can deliver fast, scalable detection of pre-catalogued patterns while also demonstrating that this capability is categorically bounded by the template ceiling. The second generation extended this with rule-based orchestration introducing primitive conditional adaptability, but remained bounded by the coverage completeness assumption. The third generation—LLM-based agents—represents a genuine qualitative advance: PentestGPT’s 67% completion on expert-level challenges, HackingBuddyGPT’s autonomous privilege escalation, and Liu et al.’s 87% CVE exploitation (Liu et al., 2024) all confirm that the template ceiling has been breached.

The critical finding is that breaching the template ceiling is not equivalent to achieving expert human performance. The gap is structural, rooted in three causal mechanisms: the information bottleneck of fixed-capacity context windows (Mechanism 1), the implicit state representation embedded in natural language history (Mechanism 2), and unverified inference propagation when the same model is responsible for reasoning and memory without independent verification (Mechanism 3).

8.2 Evaluation of Research Hypotheses

H1 – Architecture Primacy is supported. Liu et al.’s 12.4-fold performance gap between structured-context and unstructured-context conditions (Liu et al., 2024), and the 4.7-fold gap between PentestGPT and its unaugmented baseline (Deng et al., 2023), both demonstrate that architecture is the dominant performance determinant for a given model. The binding constraint is structural, not model capability.

H2 – Convergent Failure is supported. Despite architectural diversity across PentestGPT, HackingBuddyGPT, and AutoAttacker, their primary failure modes converge on one shared cause: the inability to maintain reliable, structured investigation state across sessions longer than their effective context window. This convergence is logically derivable from the shared trajectory-as-memory model.

8.3 The Research Gap Taxonomy

The six-item taxonomy—Semantic Generalisation Deficit (G1), Heuristic Brittleness (G2), Chain Blindness (G3), Stateless Reasoning (G4), Absent Hypothesis Lifecycle (G5), Safety Architecture Absence (G6)—constitutes the primary scientific contribution of this thesis. Each gap is evidentially justified, architectural in character (not closeable by quantitative improvement within existing paradigms), and appropriately scoped to guide research design. G3, G4, and G5 are identified as a compounding cluster whose combined effect exceeds the sum of individual gaps.

8.4 Research Agenda and Closing Remarks

Four research directions emerge as highest priority: (1) domain-specific external state architectures maintaining hypothesis lifecycle, payload history, and finding graphs outside the LLM context window; (2) verified confirmation standards requiring independently verifiable exploitation evidence before recording a finding; (3) a standardised penetration testing benchmark structured around the G1–G6 taxonomy with ground-truth confirmation and reproducible protocols; (4) safety-first architecture research applying formal verification to scope enforcement, prompt injection filters, and audit trail integrity.

Penetration testing automation has been a stated goal of the security community for as long as automated tools have existed. The first generation made it fast within narrow bounds; the third generation made it semantically capable within short time horizons. The next generation must make it structurally coherent across the full duration of a real engagement. That transition requires shifting the primary research focus from model capability to agent architecture: from “how smart is the model?” to “does the surrounding system give the model what it needs to be effective?” The gap taxonomy established in this thesis is offered as a starting point for that shift.

Bibliography

- Bishop, M. (2003). *Computer Security: Art and Science*. Addison-Wesley Professional.
- Deng, G., Liu, Y., Mayoral-Vilches, L., et al. (2023). PentestGPT: An LLM-empowered automatic penetration testing tool. *arXiv preprint arXiv:2308.06782*.
- Díaz-Verdejo, J., Málaga, M., Pereñíguez, F., and García-Teodoro, P. (2022). On the evaluation of dynamic analysis tools for security assessment of web applications. *IEEE Access*, 10:64874–64887.
- Greshake, K. et al. (2023). Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*.
- Happe, A. and Cito, J. (2023). Getting pwned by AI: Penetration testing with large language models. In *Proc. ESEC/FSE 2023*, pp. 2082–2086. ACM.
- IBM Security (2024). *Cost of a Data Breach Report 2024*. IBM Corporation. <https://www.ibm.com/reports/data-breach>.
- ISC2 (2023). *Cybersecurity Workforce Study 2023*. <https://www.isc2.org/research>.
- Ji, Z. et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Kennedy, D., O’Gorman, J., Kearns, D., and Aharoni, M. (2011). *Metasploit: The Penetration Tester’s Guide*. No Starch Press.
- Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS 2020*.
- Liu, N. F. et al. (2023). Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*.
- Liu, R., Yang, Z., Li, C., and Yang, H. (2024). Autonomous exploitation of one-day vulnerabilities using large language models. In *Proc. 33rd USENIX Security Symposium*.
- ProjectDiscovery (2020). *Nuclei: Fast and customisable vulnerability scanner*. <https://github.com/projectdiscovery/nuclei>.
- OWASP Foundation (2021). *OWASP Top 10 — 2021*. <https://owasp.org/Top10/>.

- OWASP Foundation (2010–present). *OWASP Zed Attack Proxy (ZAP)*. <https://www.zaproxy.org/>.
- Packer, C. et al. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
- PTES Technical Guidelines (2012). *The Penetration Testing Execution Standard*. <http://www.pentest-standard.org/>.
- Qin, Y. et al. (2023). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*.
- Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.
- Schick, T. et al. (2023). Toolformer: Language models can teach themselves to use tools. In *Proc. NeurIPS 2023*.
- Stampar, M. and the SQLMap project (2006–present). *SQLMap: Automatic SQL Injection and Database Takeover Tool*. <https://sqlmap.org/>.
- Vaswani, A. et al. (2017). Attention is all you need. In *Proc. NeurIPS 2017*.
- Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Proc. NeurIPS 2022*.
- Xu, X. et al. (2024). AutoAttacker: A large language model guided system to implement automatic cyber-attacks. *arXiv preprint arXiv:2403.01038*.
- Yao, S. et al. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Yao, S. et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS 2023*.